

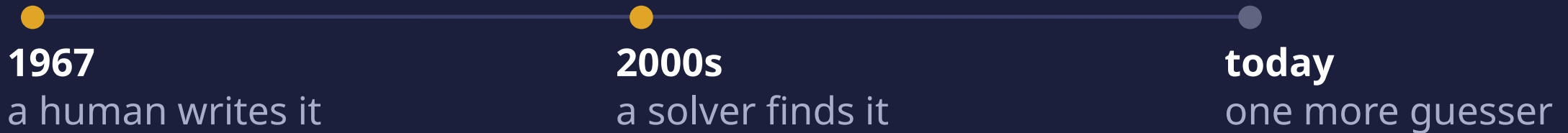
Constrained Horn Clauses and their Applications

Orna Grumberg
Technion

From Program Verification to Constrained Horn Clauses

Who Finds the Invariant?

From Floyd (1967) → Constrained Horn Clauses → LLMs



Program Verification

Given

- a program and
- a formal specification – a logical formula

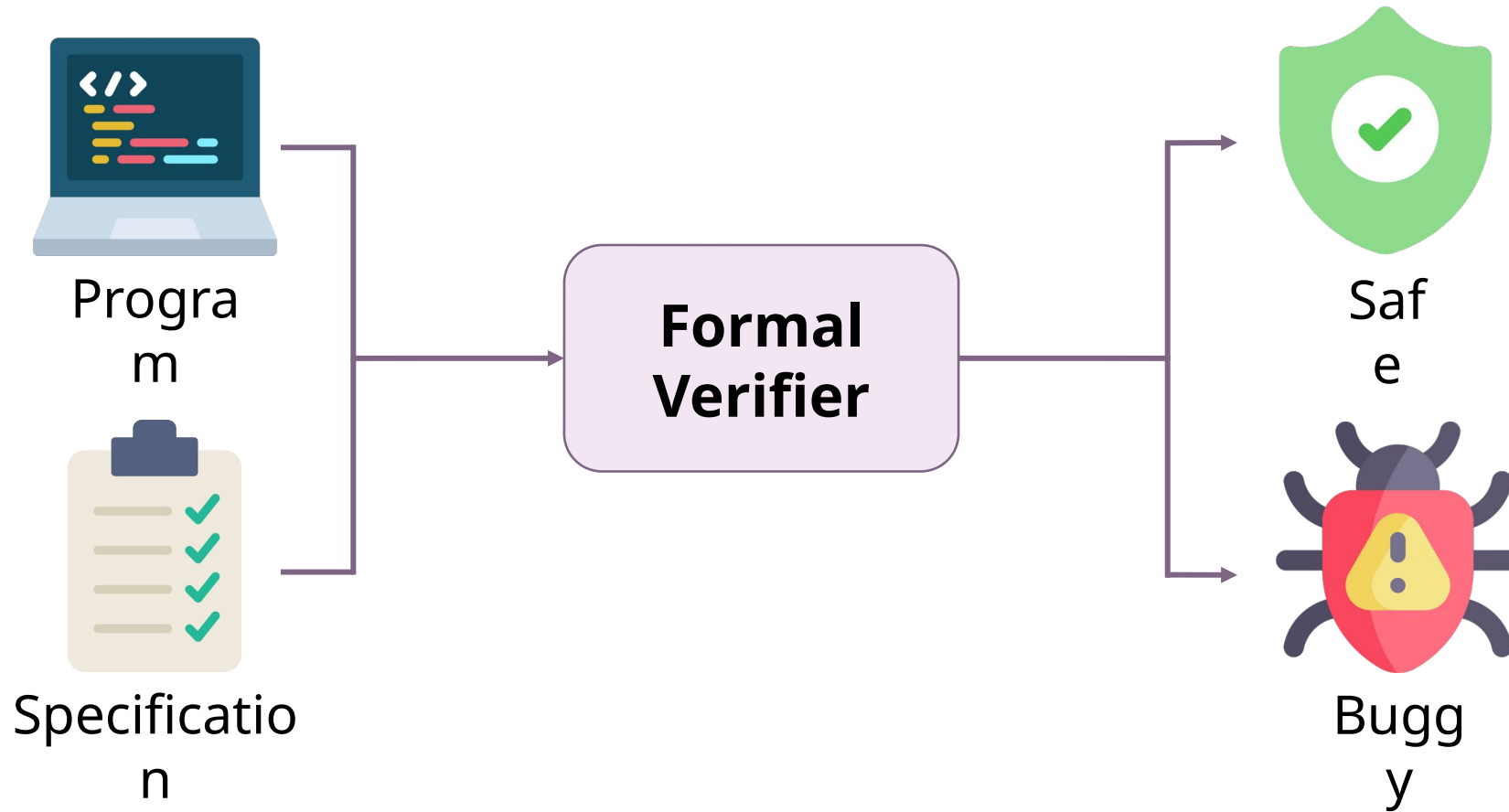
does the program satisfy the specification?

Not decidable!

- Floyd, Hoare (late 1960's) suggested methods to verify input-output specifications

Undecidable does not mean hopeless — no method is automatic, complete and always terminating.

Program Verification



An Example of Verification Problem

```
1 int f(int x) {  
2   int a = 0;  
3   int b = 0;  
4  
5   while (a < x) {  
6     a++; b--;  
7   }  
8  
9   return a*b;  
10 }
```



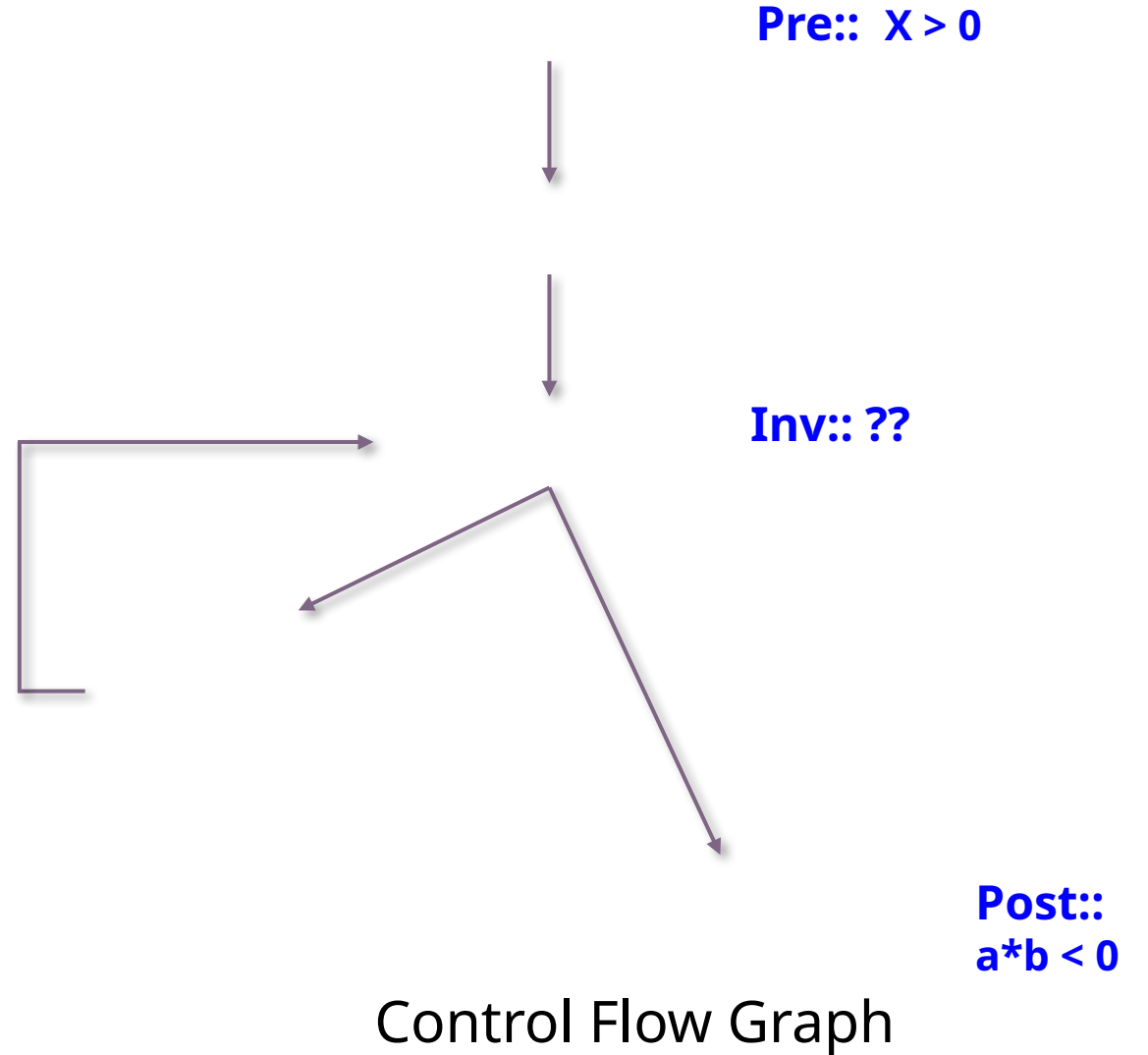
Input-output specification:

If Pre holds at the beginning, then
Post holds at the end



Program Verification (Floyd method)

```
1 int f(int x) {  
2   int a = 0;  
3   int b = 0;  
4  
5   while (a < x) {  
6     a++; b--;  
7   }  
8  
9   return a*b;  
10 }
```



Program Verification (Floyd method)

We prove:

- **Initiation:**

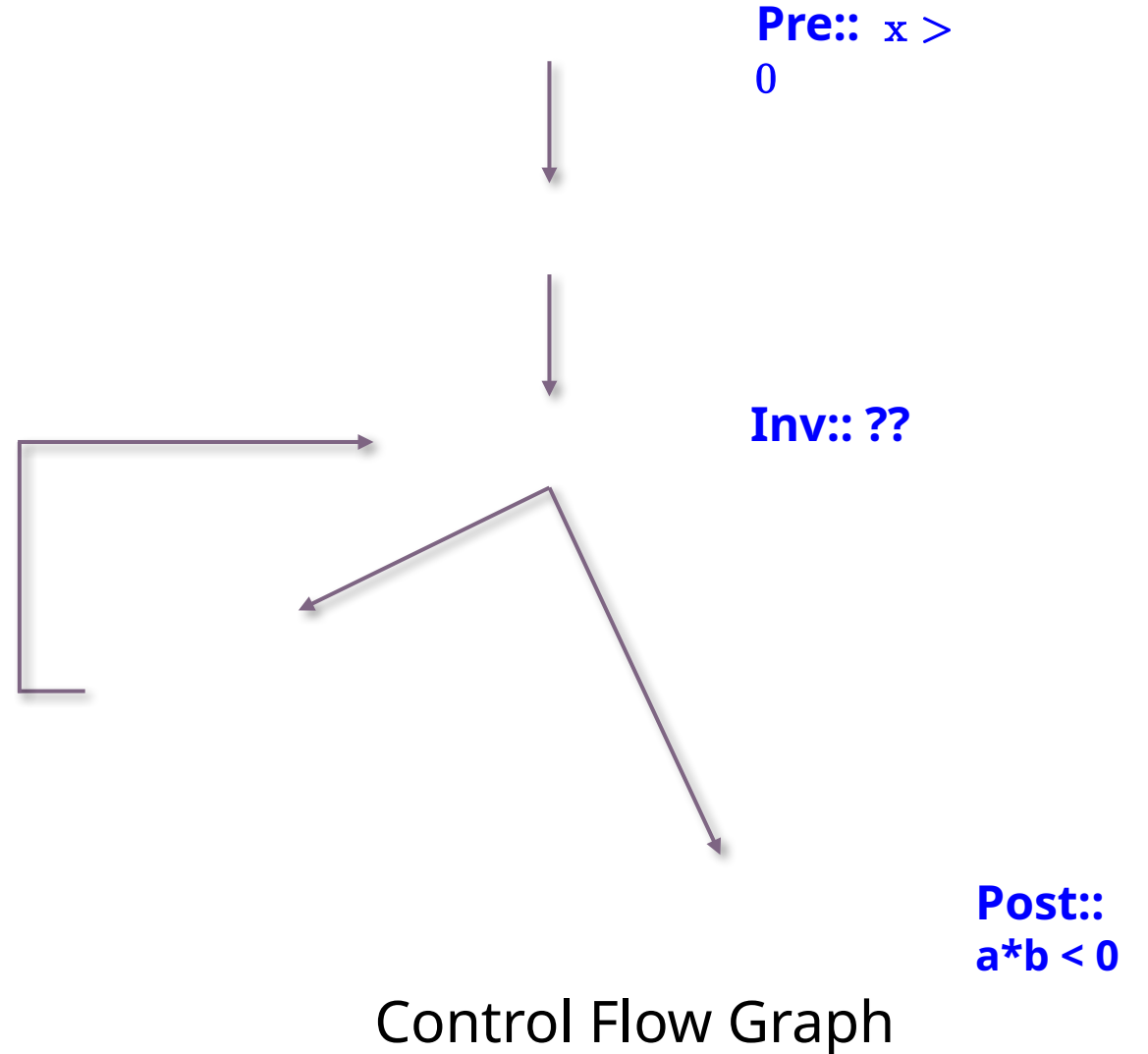
$\text{Pre}^{\wedge} (a=0 \wedge b=0) \rightarrow \text{Inv}(x, 0, 0)$

- **Consecution:**

$\text{Inv}(x, a, b) \wedge a < x \wedge a' = a+1 \wedge b' = b-1$
 $\rightarrow \text{Inv}(x, a', b')$

- **Safety:**

$\text{Inv}^{\wedge} \neg(a < x) \rightarrow \text{Post}$



Program Verification (Floyd method)

We prove:

- **Initiation:**

$\text{Pre}^{\wedge} (a=0 \wedge b=0) \rightarrow \text{Inv}(x, 0, 0)$

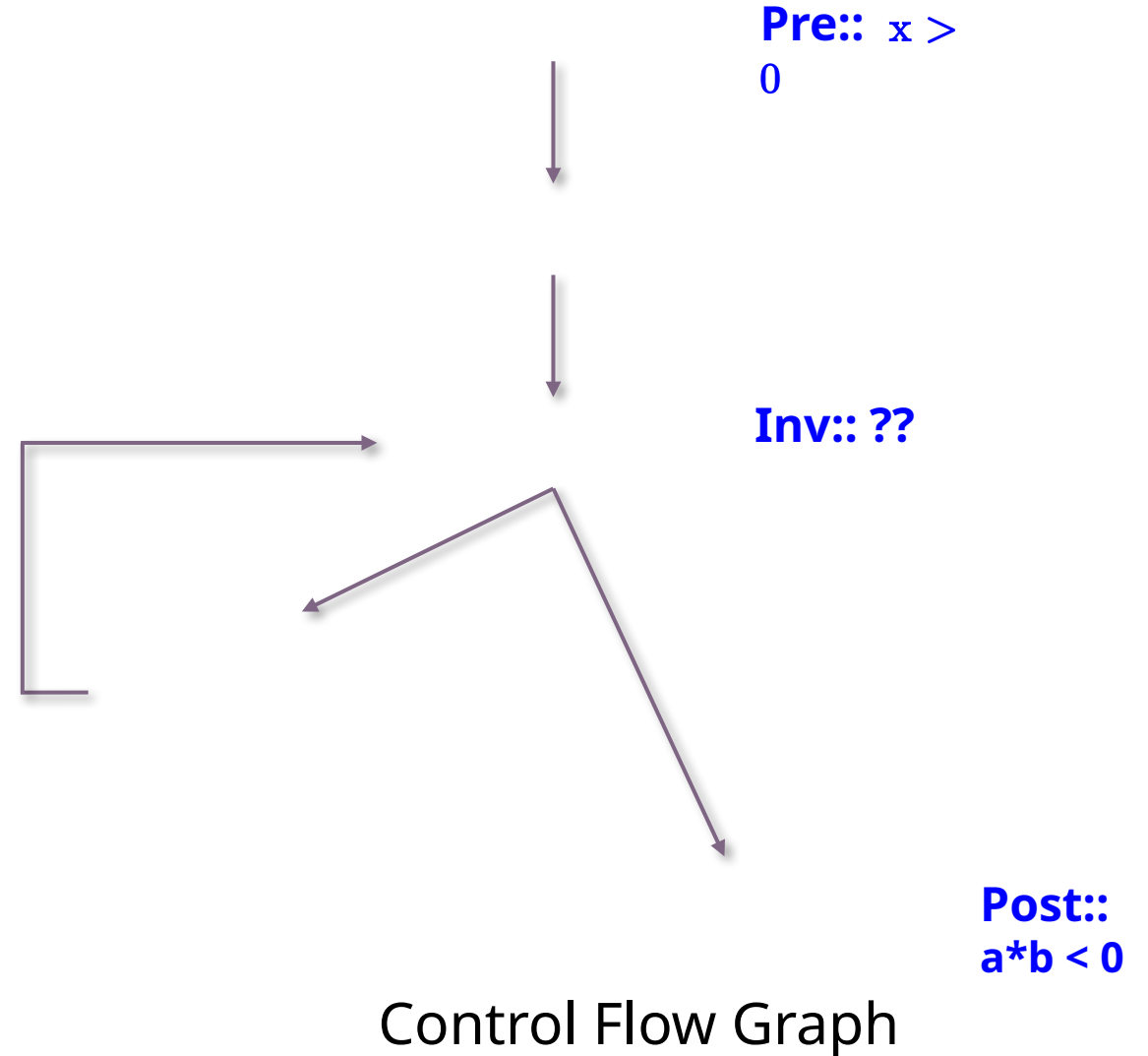
- **Consecution:**

$\text{Inv}(x, a, b) \wedge a < x \wedge a' = a+1 \wedge b' = b+1$
 $\rightarrow \text{Inv}(x, a', b')$

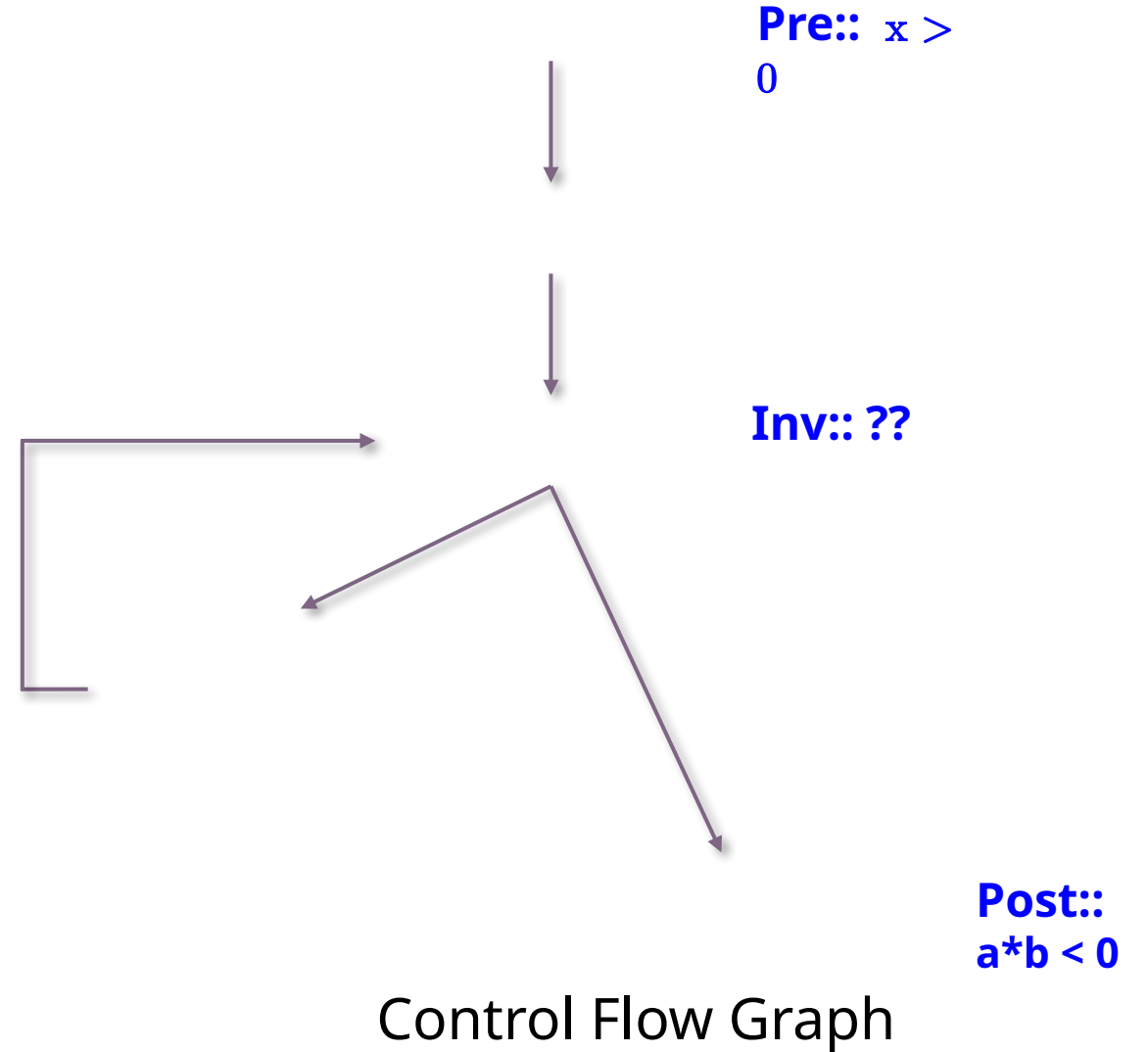
- **Safety:**

$\text{Inv} \wedge \neg(a < x) \rightarrow \text{Post}$

Is **$a+b=0$** a good inductive invariant?



Program Verification (Floyd method)



Let the machine find the invariant for us

- Model checking, symbolic verification, and Constrained Horn Clauses

Two ways to make it automatic

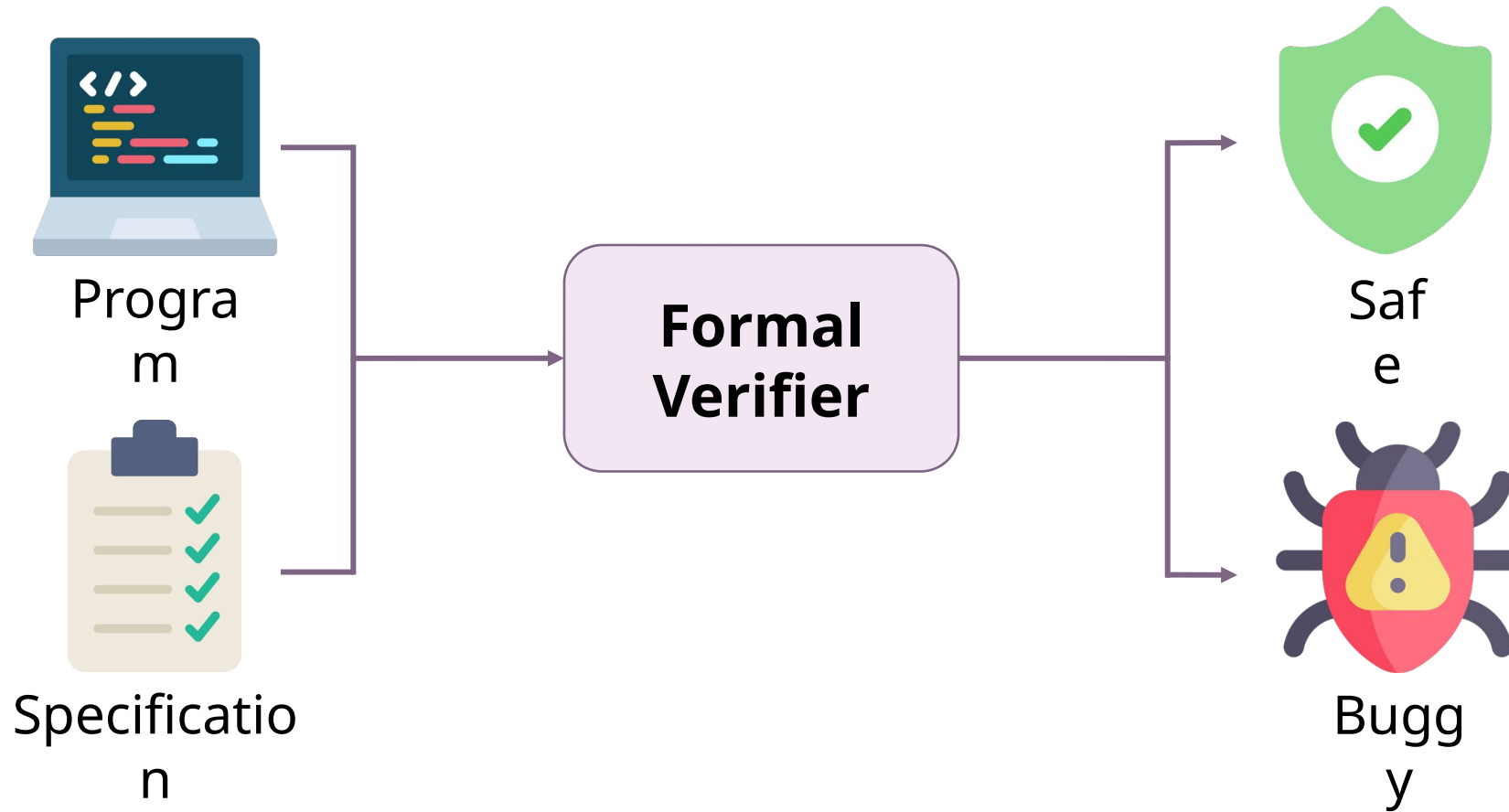
Restrict until decidable

- **Model checking** (Clarke, Emerson, Sifakis, early 1980s)
 - finite-state model + temporal-logic property
 - answers yes, or no with a counterexample
- **But programs are not finite state**

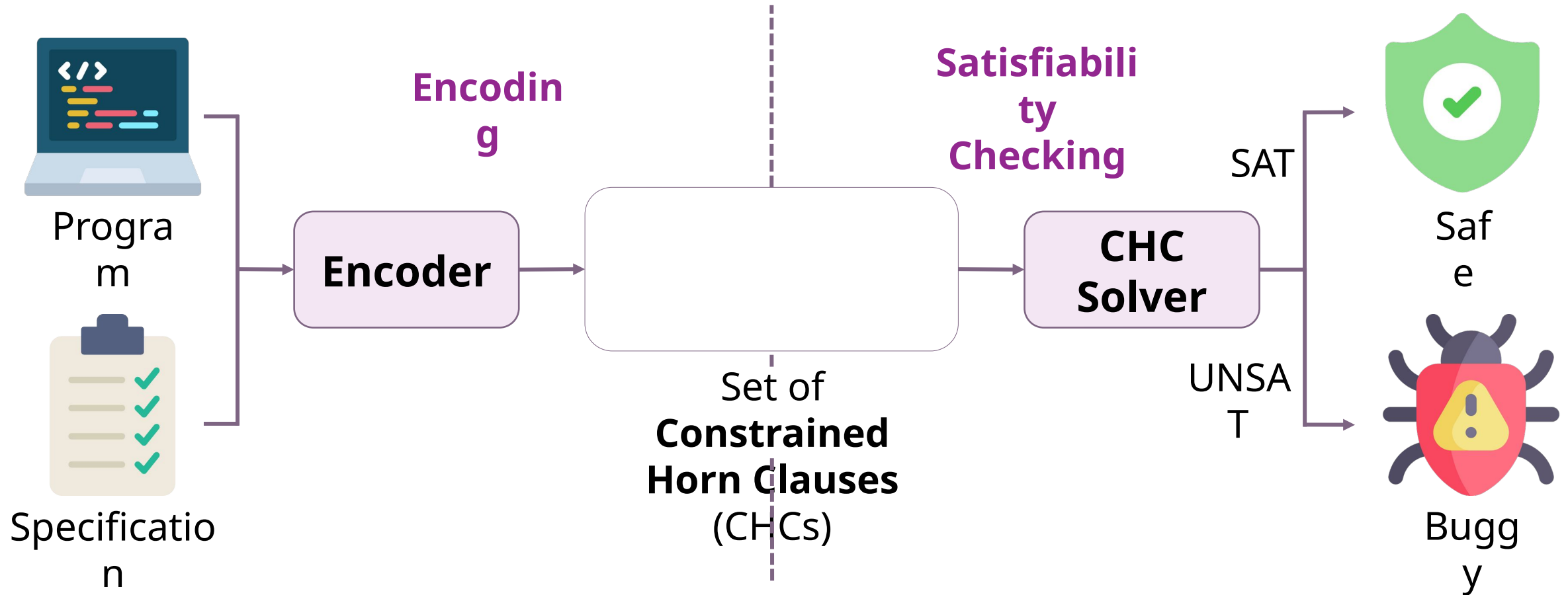
Keep programs — hand it to a solver

- Represent the program and the specification **symbolically** — as logical formulas.
- Reduce verification to:
is this formula satisfiable?
- Let **very strong solvers** answer the question.
- **SAT → SMT → CHCs**
 - **the invariant becomes the unknown that the solver is asked to find**

Back to Program Verification

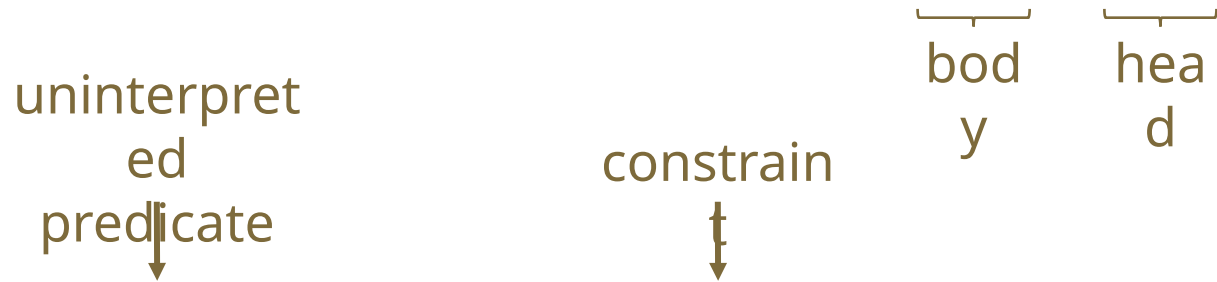


Program Verification via CHCs



Constrained Horn Clauses (CHCs)

-



An Example Verification Problem

```
1 bool f(int x) {  
2   int a = 0;  
3   int b = 0;  
4  
5   while (a < x) {  
6     a++; b--;  
7   }  
8  
9   if (a*b < 0) {  
10    return true;  
11  }  
12  return false;  
13 }
```

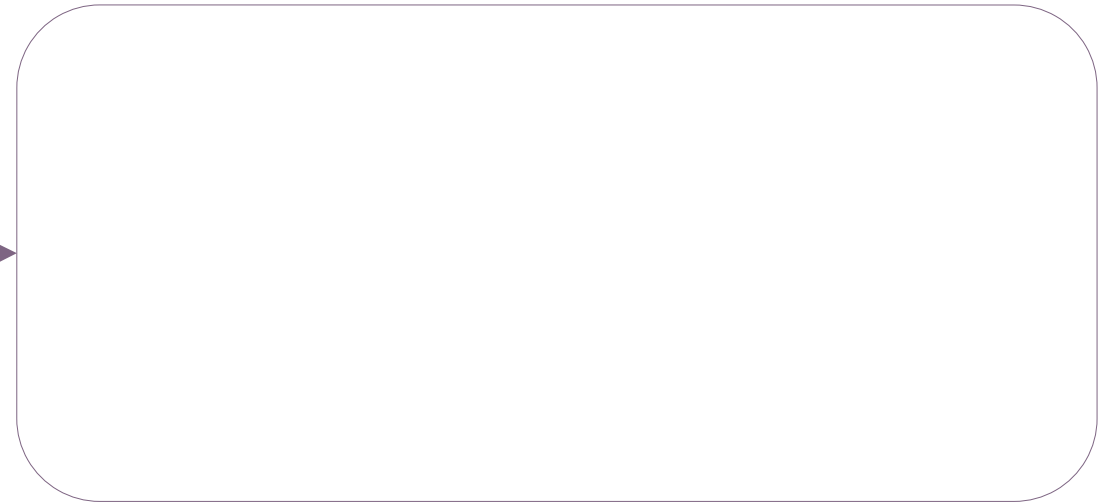
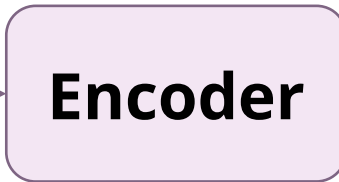


An Example Verification Problem

Encoding



```
1 bool f(int x) {  
2   int a = 0;  
3   int b = 0;  
4  
5   while (a < x) {  
6     a++; b--;  
7   }  
8  
9   if (a*b < 0) {  
10    return true;  
11  }  
12  return false;  
13 }
```



Set of CHCs



Derivations



Derivation

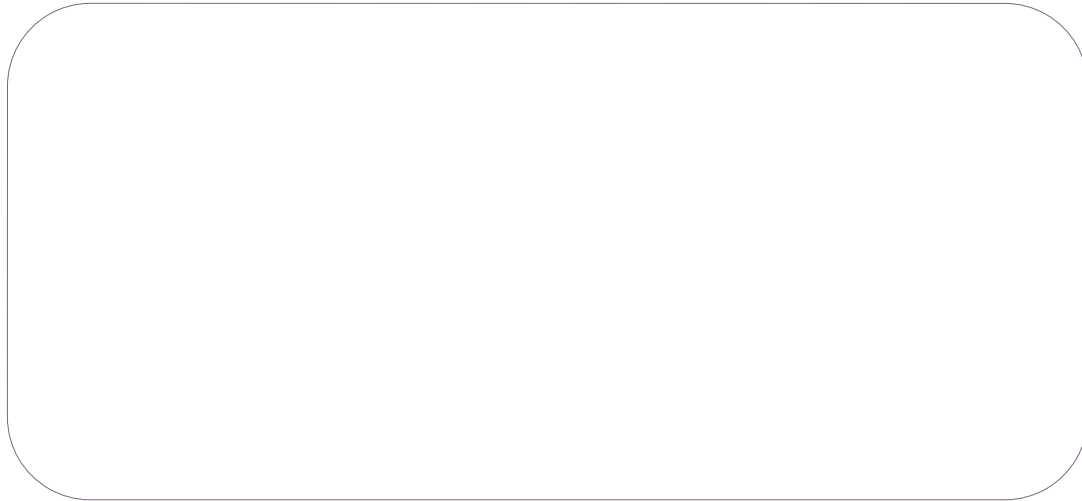
**Query
Check**



Proving Satisfiability with Interpretations

Enumerating all derivations is usually infeasible.

Instead, CHC solvers search for symbolic **interpretations** of the uninterpreted predicates that satisfy all CHCs.



Satisfying Interpretation

$I(p)$ is an inductive
invariant!

THE SAME OBJECT, SIXTY YEARS APART

Floyd's obligations are the three CHCs

1967 · the user supplies Inv

Pre $\wedge$ init $\rightarrow$ **Inv**

Inv $\wedge$ guard $\wedge$ body $\rightarrow$ **Inv'**

Inv $\wedge$ $\neg$ guard $\rightarrow$ **Post**

today · the solver supplies p

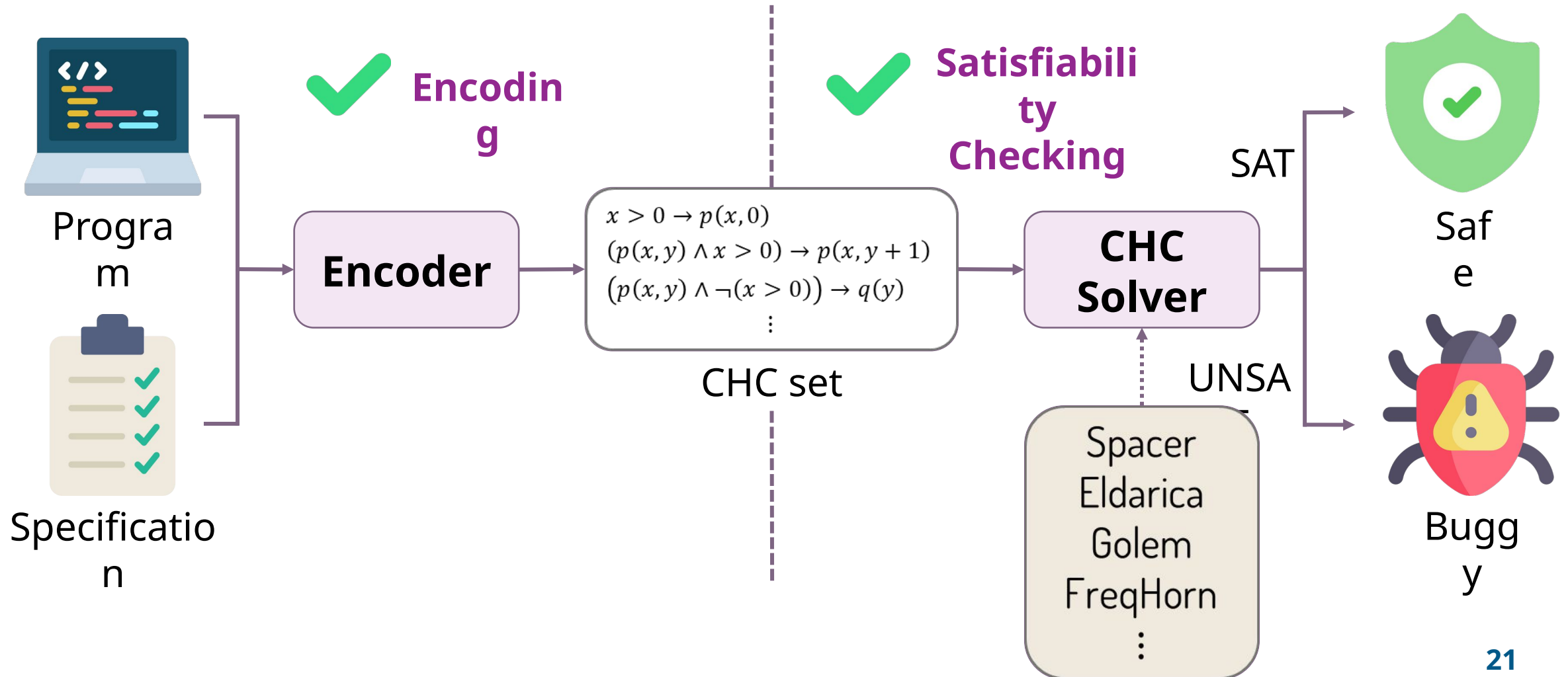
x>0 $\wedge$ a=0 $\wedge$ b=0 $\rightarrow$ **p(x, a, b)**

p(x, a, b) $\wedge$ a<x $\rightarrow$ **p(x, a+1, b-1)**

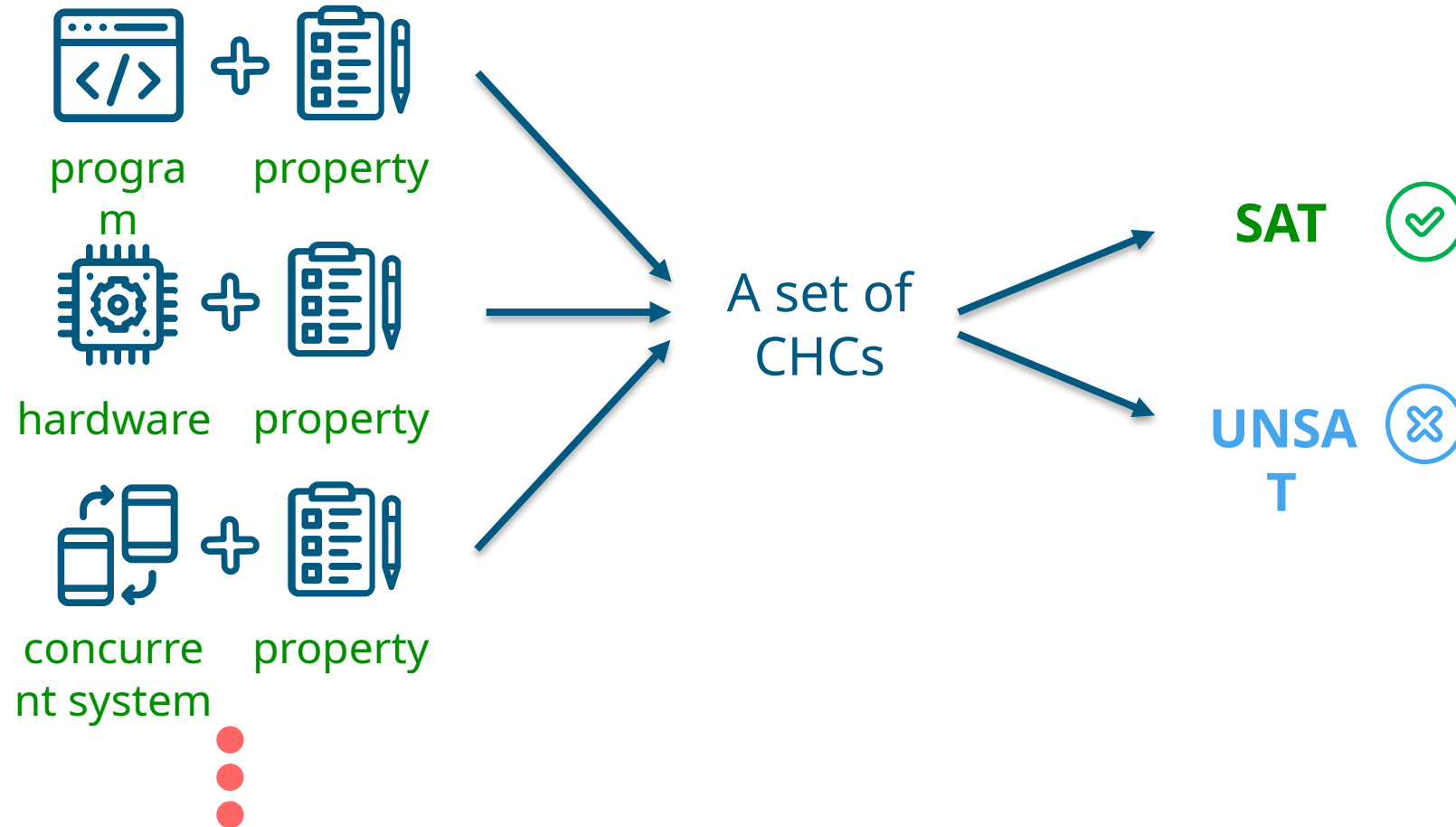
p(x, a, b) $\wedge$ $\neg$ (a<x) $\rightarrow$ **a*b<0**

Same three obligations — only who produces the invariant changed.

Program Verification via CHCs



Applications



BE HONEST ABOUT THIS

What FM guarantees — and what it does not

SAT → meets its spec on all behaviors. UNSAT → a bug, with a counterexample.

Guaranteed — but only relative to three things:

The encoding

does the model say
what the code does?

The specification

is it the property we
wanted?

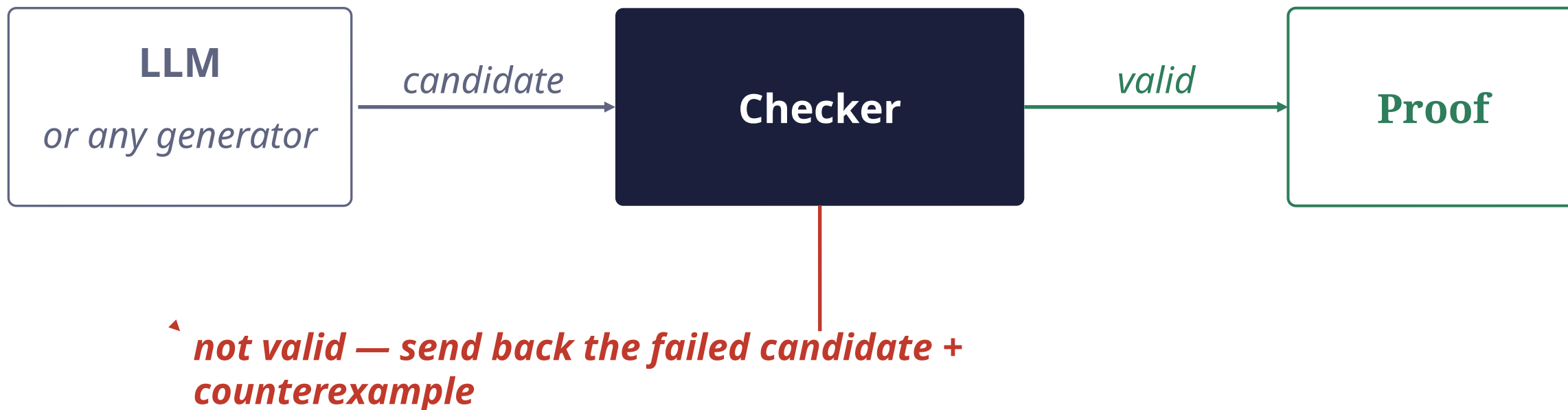
Termination

CHC solving is
undecidable — it may
never answer

Exactly where an LLM can help — and where it must not be trusted!

Can LLM help us find invariants?

The loop, concretely



- The dark box is the central building block. The pale box is replaceable.
- Not valid → the same information Floyd's checks gave a human.

Conclusion

- This talk presents a very limited view of what formal verification is:
an invariant finding for proving a program correctness
 - Floyd method and solvers can handle program with much richer structures
 - More complex specifications
- Recent works apply LLM in different ways to assist formal methods
 - NOT to replace them
 - They do not guarantee correctness unless they are checked by a formal checker

Questions?

